

Final Capstone Project: Customer Churn Model Deployment

Module 8 Assignment Submission & Project Overview

GitHub Repository URL:	https://github.com/Moksh-72/103_Final.git
Project Folder:	customer-churn-api/
Framework / Server:	Flask WSGI Server (Python 3.x)
Model Architecture:	Logistic Regression (ROC-AUC: 0.8481, Accuracy: 80.72%)
Batch Scoring Input:	test_data/all_customers.csv (7,043 records)
Batch Outputs:	scored_customers.csv & logs/batch_log.txt

1. Executive Summary

This project completes the end-to-end deployment of a Machine Learning customer churn prediction model. The model has been packaged into a production-grade HTTP REST API using Flask and integrated into an automated nightly batch scoring pipeline. All outputs, prediction probabilities, and runtime exceptions are tracked using Python's standard logging framework. The repository includes a complete maintenance strategy outlining retraining triggers, feature drift detection, and semantic versioning.

2. System Architecture & Implementation

A. Real-Time Inference API (app/main.py)

The model is hosted as a Flask web application on port 8000. It exposes a POST /predict endpoint that accepts customer attribute JSON payloads, passes them through a pre-fitted ColumnTransformer (transformer.pkl), and computes churn predictions using the trained model (model.pkl). It returns probability and binary predictions in JSON format:

Sample Request Payload:

```
{"customer": {"gender": "Female", "SeniorCitizen": 0, "Partner": "Yes", "tenure": 1, "MonthlyCharges": 29.85, "TotalCharges": 29.85, ...}}
```

API JSON Response:

```
{"churn_prediction": "Yes", "churn_probability": 0.6373}
```

B. Automated Batch Scoring Pipeline (batch.py)

The batch scoring pipeline accepts --input test_data/all_customers.csv, sends each customer row as JSON to the API endpoint over HTTP, and compiles all results into scored_customers.csv. High-throughput multithreading allows scoring all 7,043 customers in seconds while generating structured logs in logs/batch_log.txt.

3. Batch Execution & Monitoring Metrics

Metric Name	Execution Result
Total Customer Records Processed	7,043
Successful Predictions	7,042 (99.986%)
Failed Requests / Exceptions	1 (Tracked in logs/batch_log.txt)

Average Churn Probability	0.2654 (26.54%)
Primary Log File Path	customer-churn-api/logs/batch_log.txt

4. Model Maintenance Plan Overview

Retraining Strategy: Models undergo monthly scheduled retraining on fresh 30-day active data. Event-driven retraining is triggered if ROC-AUC drops below 0.75 or accuracy falls below 78%. Automated evaluation gates require candidate models to surpass baseline performance by 1.5% ROC-AUC before blue-green deployment.

Drift Detection: Numerical features (tenure, MonthlyCharges, TotalCharges) are monitored nightly using Population Stability Index (PSI) and KS-tests. PSI > 0.25 flags data drift; rolling 7-day average churn probability shifts > 15% trigger alerts.

Versioning & Documentation: Semantic Versioning (vMAJOR.MINOR.PATCH) governs releases. Model artifacts are tracked in MLflow alongside Model Cards detailing dataset checksums, hyperparameters, and ROC-AUC baselines.

5. Repository Directory Layout

```
customer-churn-api/
├── app/
│   ├── __init__.py      # Package initializer
│   ├── main.py          # Flask API server (python -m app.main)
│   ├── model.pkl        # Trained Logistic Regression model
│   ├── transformer.pkl   # Scikit-learn ColumnTransformer
│   └── utils.py          # Inference helper functions
├── test_data/
│   ├── sample_input.json # Test JSON payload for /predict
│   └── all_customers.csv  # Customer dataset for batch scoring
├── logs/
│   └── batch_log.txt      # Detailed execution & monitoring logs
├── batch.py              # Batch scoring pipeline script
├── requirements.txt       # Python dependency manifest
├── README.md             # Project README & Maintenance Plan
└── .gitignore            # Git ignore directives
```